AI-Powered Student Project Discovery Platform

Project Ahaa - Detailed Technical Report

This report explains what the project is, what it does, which packages and libraries it uses, why those tools are chosen, and how the architecture is organized.

1. Introduction

Project Ahaa is an AI-powered web-based platform developed to help students discover existing project ideas, compare them with new ideas, and get guidance on improving the project before implementation. The core aim is to reduce duplication, increase creativity, and make project planning easier for students.

In simple terms, the project acts like a smart project discovery assistant. A user can enter a concept or idea, and the system helps by suggesting similar projects, recommending technologies, improving the idea, showing related research papers, and building a knowledge graph of the project landscape.

2. What This Project Is Doing

The project is doing three major jobs at the same time:

- It helps users discover similar or related projects from a collection of stored project records.
- It provides AI-assisted recommendations such as improvements, technologies, and research papers.
- It visualizes relationships between projects, technologies, and domains through a knowledge graph and timeline.

This means the project is not just a search tool. It is a complete project discovery and planning platform that supports idea generation, evaluation, and exploration.

3. Main Features of the System

- Project search and discovery: users can browse projects stored in the system.
- Similarity matching: similar projects are found using text embeddings and cosine similarity.
- Idea improvement: the system suggests how a student project idea can be improved or extended.
- Technology recommendations: it suggests suitable frameworks, tools, and libraries for the idea.
- Research paper suggestions: it connects the idea to academic papers through external APIs.
- Knowledge graph visualization: it creates a graph of projects, technologies, and domains.
- Timeline view: it shows how related project ideas evolved over time.

4. Technologies and Libraries Used

The project uses a combination of frontend, backend, data science, and visualization libraries. Each library is chosen to support a specific purpose.

Frontend libraries:

- React: used to build the interactive user interface.
- React DOM: used to render the React application in the browser.
- Axios: used to communicate with the backend API from the frontend.
- React Scripts: used to run and build the React application easily.

Backend libraries:

- FastAPI: used to create the API endpoints and handle HTTP requests efficiently.
- Uvicorn: used as the ASGI server to run the FastAPI app.
- Pydantic: used for request validation and schema modeling.
- Requests: used to call external APIs such as Semantic Scholar and arXiv.

AI and data processing libraries:

- NumPy: used for numerical operations, especially vector and matrix calculations.
- Scikit-learn: used for cosine similarity and vector-based matching.
- Sentence Transformers: used to generate text embeddings that represent the meaning of project descriptions.
- NetworkX: used to build and manage the knowledge graph structure.

Why these libraries are used:
The frontend libraries are chosen because the application is a modern web app that needs a smooth and responsive UI. The backend libraries are chosen for speed, simplicity, and ease of API development. The AI libraries are chosen because the project depends on text understanding, semantic similarity, and graph-based relations.

5. Project Architecture

The system follows a simple three-layer architecture:

- Presentation layer: the React frontend provides all user interaction and visualization.
- Application layer: the FastAPI backend processes requests and connects the frontend to the AI modules.
- Data layer: JSON files store project records and the technology knowledge base.

The architecture is modular. Each feature is placed in a separate Python module so that the system is easier to maintain and expand. The API layer acts as the central coordinator that connects user requests to the appropriate logic module.

6. How the System Works

A typical workflow is as follows:

- The user opens the app and enters a project idea or keyword.
- The frontend sends the request to the FastAPI backend.
- The backend calls the relevant function such as similarity search or idea improvement.
- The AI module processes the input and returns recommendations or graph data.
- The frontend displays the results in a user-friendly view.

For example, when a student searches for a project idea, the system converts the request into a vector representation, compares it with existing project data, and returns the most similar entries. This makes it easier to see whether the idea already exists or whether it can be improved.

7. Why the Project Is Designed This Way

The design is simple because the project is mainly a prototype and academic demonstration platform. A lightweight architecture is better here than a large production system because it makes the project easier to understand, run, and improve. The use of JSON files instead of a full database keeps setup easy and suitable for local development.

The project also uses fallback logic for recommendations. This means that even if an external API is unavailable, the app can still provide useful outputs through local rules or keyword matching. That improves robustness and makes the system more practical for students.

8. Strengths of the Project

- Clear separation between frontend, backend, and AI logic.
- User-friendly interface for project discovery.
- Good support for idea improvement and guidance.
- Can function even when external APIs are limited.

9. Possible Improvements

The project can be improved further by integrating a real database, using environment variables for secrets, improving the UI design, and adding user authentication. These improvements would make the platform more scalable and suitable for real-world deployment.

10. Conclusion

Project Ahaa is a smart project discovery and idea planning platform that combines AI, web development, and data visualization. It is designed to assist students by helping them discover similar projects, improve ideas, find relevant technologies, and explore research literature. The architecture is modular, practical, and well suited for educational and prototype use.

11. Key Project Files

File	Role
backend/main.py	Main FastAPI application and API routes
backend/similarity_engine.py	Similarity search using embeddings
backend/idea_improver.py	Improvement suggestions for project ideas
backend/tech_recommender.py	Technology recommendations
backend/paper_recommender.py	Research paper suggestions
backend/knowledge_graph.py	Knowledge graph construction
frontend/src/App.js	Main React application interface